

AAD- Arquiteturas de Alto Desempenho

Relatório Projeto 1 “Mining DETI Coins”

Miguel Vila (miguelovila@ua.pt), 107276

Matilde Teixeira (matilde.teixeira@ua.pt), 108193

1. Introdução

Este relatório apresenta uma análise detalhada das diferentes implementações desenvolvidas para a mineração de DETI Coins, focando nos aspectos de desempenho e nas características que afetam a eficiência de cada abordagem.

2. Análise das Implementações

Este projeto apresenta uma progressão natural de otimizações, começando com uma implementação básica e evoluindo para versões mais sofisticadas. Dada a quantidade de combinações de implementações, a seguinte tabela permite perceber com facilidade o que foi implementado.

	CPU	AVX	AVX2	AVX512F	NEON	CUDA	WebAssembly
Single Thread	Sim	Sim	Sim	Sim	Sim	N.A.	Sim
Multi Thread (OMP)	Sim	Sim	Sim	Sim	*	N.A.	N.A.
Cliente Servidor	Sim	Sim	Sim	Sim	*	Sim	N.A.
Pesquisa por Palavra	Não	Não	Sim	Não	*	Não	N.A.

Tabela 2.1 - Implementações desenvolvidas

De igual forma organizámos o número de tentativas realizadas em 120 segundos na próxima tabela. Os testes foram realizadas numa máquina com um processador Intel i7-7700HQ(8 cores)@3.800GHz e placa gráfica NVIDIA GeForce GTX 1050 Mobile. Apenas o teste AVX512F foi feito com outro computador que suportasse essas instruções Ryzen 7 7745HX(16 cores). O número de random words foi deixado no valor padrão (1). A implementação NEON foi testada numa VPS com um processador Ampere A1 Compute (4 cores).

	CPU	AVX	AVX2	AVX512F	NEON	CUDA
Single Thread	1.1715×10 ⁹	3.3672×10 ⁹	6.1125×10 ⁹	1.6198×10 ¹⁰	9.4397×10 ⁸	5.4459×10 ¹¹

Tabela 2.2 - Número de tentativas por 120 segundos

A implementação em WebAssembly também foi testada por 120 segundos em dois dispositivos diferentes:

- Zen Browser (AMD Ryzen™ 7 5800H): 1×10⁹ tentativas em 57.860 segundos;
- Google Pixel 8 (Tensor G3): 1×10⁹ tentativas em 148.109 segundos.

* sem acesso consistente a um computador que suporte instruções NEON.

2.1. Aspectos Interessantes

2.1.1. Pesquisa por Palavra

A pesquisa por palavra é uma variante especializada do algoritmo AVX2 com paralelização, desenvolvida para encontrar DETI Coins que contenham uma estrutura com o formato padrão "DETI coin", seguida da sequência de pesquisa e de uma palavra aleatória de 4 bytes (garante que os search spaces são diferentes entre as lanes e threads) Este é um exemplo do seu formato:

```
'DETI coin string_a_procurar&G#n \n'
```

Como o objetivo é pesquisar pela DETI Coin com a string é importante que quando o espaço de pesquisa seja exausto a string não se altere, logo, sempre que é verificada essa situação a palavra random é novamente gerada e o resto da moeda reposta a ' '. No entanto, devido às limitações do formato da moeda (52 bytes no total), a utilização de sequências de pesquisa mais longas (até 36 caracteres permitidos) reduz significativamente o espaço disponível para variações, resultando num maior número de reinicializações e, conseqüentemente, numa diminuição do desempenho do algoritmo. Pesquisa de 120 segundos com `deti_coins_cpu_avx2_omp_special_search()` por:

- "Arquiteturas Alto Desempenho 24/25!!" : 7.13487416×10^8 tentativas;
- "AAD!" : $7.5383264304 \times 10^{10}$ tentativas.

2.1.2. Impacto de Operações de I/O

Um dos aspectos mais significativos que afetam o desempenho é a inclusão de operações de I/O dentro dos loops principais. Por isso mesmo, apenas as incluímos quando o programa é compilado com a flag DEBUG. Em 120 segundos de AVX2 OMP:

- `DEBUG=1` : 1.3559×10^{10} tentativas;
- `DEBUG=0` : 7.9936×10^{10} tentativas.

```
while (!stop_request) {
    md5_cpu_avx2((v8si *)coin_data, (v8si *)hash_data);

    for (u32_t lane = 0; lane < 8u; lane++) {
        (...)
        #if DEBUG > 0
            printf("Thread %02d:\n", thread_id);
            print_coin_in_lane(coin_data, lane, 8);
            printf("\n");
        #endif
        (...)
    }
}
```

3. Sobre Algumas Implementações

3.1. Implementação Cliente-Servidor

A arquitetura cliente-servidor desenvolvida distribui o processo de mineração de DETI Coins através de múltiplas máquinas em rede. No protocolo de comunicação que criámos, existem 3 tipos de mensagem enviadas com *sockets*: HELLO, CONFIG e COIN_FOUND. O fluxo de informação processa-se da seguinte forma:

1. O programa é lançado em modo orquestrador (-o) com um determinado `n_random_words`;
2. O programa lançado como client (-c*) durante x segundos e conecta-se ao servidor enviando uma mensagem HELLO com o seu hostname, tecnologia que está a usar e número de threads;
3. O servidor responde numa mensagem CONFIG com o valor `n_random_words`;
4. O cliente começa a minerar e sempre que encontrar uma moeda envia-a numa mensagem COIN_FOUND.

```
initializing a search as a mine orchestrator on 127.0.0.1:3000.
[INF] Server is listening on port 3000
[INF] Configuration: n_random_words = 4
[INF] Client proxmox just connected. Using AVX. (1 thread)
[INF] Client lartaguixa just connected. Using AVX with OpenMP (16 threads)
[INF] Received coin from proxmox
[INF] Received coin from lartaguixa
[INF] Client miguelovila just connected. Using AVX2 Special Search with OpenMP (16 threads)
[INF] Received coin from miguelovila
[INF] Received coin from lartaguixa
[INF] Received coin from miguelovila
[INF] Client proxmox disconnected
```

Um aspecto relevante desta implementação é a sua capacidade de gerir vários clientes utilizando diferentes tecnologias de mineração em simultâneo.

3.2. Implementações OpenMP

Utilizando o OpenMP implementamos paralelismo nos vários algoritmos de pesquisa (cpu, avx, avx2, avx512f e special_avx2), aproveitando vários núcleos do processador. A inicialização das *lanes* é feita dentro de uma seção crítica, garantindo que cada thread é gerada baseada numa *seed* diferente, para aumentar a aleatoriedade entre estas. Outra seção crítica é a função `save_deti_coin()`, protegida da mesma forma.

```
#pragma omp critical
{
    save_deti_coin(coin);
}
```

3.3. Implementações SIMD e Algoritmos de Pesquisa

AVX, AVX2 e AVX512 seguem uma estrutura semelhante, diferindo principalmente na quantidade de dados que conseguem processar em paralelo. A implementação AVX permite processar 4 hashes em simultâneo, a AVX2 aumenta esta capacidade para 8 hashes, e a AVX512 atinge 16 hashes em paralelo. Esta diferença pode-se ver na definição dos tipos vetoriais utilizados.

```
// AVX: 4 Lanes
typedef int v4si __attribute__((vector_size(16)));

// AVX2: 8 Lanes
typedef int v8si __attribute__((vector_size(32)));

// AVX512: 16 Lanes
typedef int v16si __attribute__((vector_size(64)));
```

O algoritmo de pesquisa implementado em todas as versões SIMD (AVX, AVX2 e AVX512) segue uma sequência lógica e repetitiva. O processo começa com a inicialização de cada uma das *lanes* em paralelo, onde é aplicado o template base "DETI coin" seguido por um número configurável de palavras aleatórias definido por *n_random_words*. O ciclo principal de pesquisa consiste em três operações fundamentais: o cálculo paralelo dos hashes MD5 para todas as lanes simultaneamente, a verificação da validade de cada hash obtido (procurando pelos zeros necessários), e o incremento sistemático do espaço de pesquisa. Este ciclo repete-se continuamente até ser encontrada uma moeda válida ou até ser necessário reinicializar o espaço de pesquisa.

A principal diferença entre as implementações reside na forma como os dados são organizados na memória.

3.4. WebAssembly

A implementação em WebAssembly para a mineração de DETI Coins segue uma abordagem simplificada praticamente idêntica à implementação CPU original, começando com a inicialização do template padrão da DETI Coin e percorrendo o espaço de pesquisa até encontrar uma hash válida (`hash[3]=0` termina com 32 zeros).

3.5. CUDA

Esta abordagem consegue atingir um desempenho bastante superior ao de qualquer outra implementação. A implementação baseia-se num kernel CUDA que organiza a computação em três níveis de paralelismo. O kernel é executado com uma configuração que cria $1024 \times 100 \times 128$ threads simultâneas, onde cada thread processa 95 tentativas diferentes de moedas.

Os resultados obtidos através da implementação CUDA demonstram um avanço significativo no desempenho da mineração de DETI Coins.